

```
In [4]: X = dataset[['Age', 'EstimatedSalary']].values
        y = dataset['Purchased'].values
```

```
In [5]: from sklearn.model_selection import train_test_split

        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.25, random_state=0
        )
```

```
In [6]: from sklearn.preprocessing import StandardScaler

        sc = StandardScaler()
        X_train = sc.fit_transform(X_train)
        X_test = sc.transform(X_test)
```

```
In [7]: from sklearn.linear_model import LogisticRegression

        classifier = LogisticRegression(random_state=0)
        classifier.fit(X_train, y_train)
```

```
Out[7]: ▼      LogisticRegression ⓘ ?
        LogisticRegression(random_state=0)
```

```
In [8]: y_pred = classifier.predict(X_test)
        y_pred
```

```
Out[8]: array([0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
                0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
                1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
                0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1,
                0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1])
```

```
In [9]: from sklearn.metrics import confusion_matrix

        cm = confusion_matrix(y_test, y_pred)
        cm
```

```
Out[9]: array([[65,  3],
               [ 8, 24]])
```

```
In [12]: TN, FP, FN, TP = cm.ravel()
```

```
In [14]: print("True Positives (TP):", TP)
        print("False Positives (FP):", FP)
        print("True Negatives (TN):", TN)
        print("False Negatives (FN):", FN)
```

```
True Positives (TP): 24
False Positives (FP): 3
True Negatives (TN): 65
False Negatives (FN): 8
```

```
In [15]: accuracy = (TP + TN) / (TP + TN + FP + FN)
print("Accuracy:", accuracy)
```

Accuracy: 0.89

```
In [16]: error_rate = 1 - accuracy
print("Error Rate:", error_rate)
```

Error Rate: 0.10999999999999999

```
In [17]: precision = TP / (TP + FP)
print("Precision:", precision)
```

Precision: 0.8888888888888888

```
In [18]: recall = TP / (TP + FN)
print("Recall:", recall)
```

Recall: 0.75

```
In [19]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
```

```
In [20]: dataset = pd.read_csv("Social_Network_Ads.csv")

X = dataset[['Age', 'EstimatedSalary']].values
y = dataset['Purchased'].values
```

```
In [21]: test_sizes = [0.1, 0.2, 0.25, 0.3, 0.4]
```

```
In [24]: results = []

for test_size in test_sizes:

    # Split the dataset
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=0
    )

    # Feature scaling
    sc = StandardScaler()
    X_train = sc.fit_transform(X_train)
    X_test = sc.transform(X_test)

    # Train Logistic Regression
    classifier = LogisticRegression(random_state=0)
    classifier.fit(X_train, y_train)

    # Prediction
    y_pred = classifier.predict(X_test)

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    TN, FP, FN, TP = cm.ravel()
```

```
# Accuracy calculation
accuracy = (TP + TN) / (TP + TN + FP + FN)

# Store results
results.append([test_size, TN, FP, FN, TP, accuracy])
```

```
In [25]: results_df = pd.DataFrame(
    results,
    columns=['Test Size', 'TN', 'FP', 'FN', 'TP', 'Accuracy']
)

results_df
```

```
Out[25]:
```

	Test Size	TN	FP	FN	TP	Accuracy
0	0.10	31	1	1	7	0.950000
1	0.20	57	1	5	17	0.925000
2	0.25	65	3	8	24	0.890000
3	0.30	74	5	11	30	0.866667
4	0.40	93	8	15	44	0.856250

```
In [26]: #change random state variable
```

```
In [27]: random_states = [0, 1, 5, 10, 20]
test_size = 0.25
```

```
In [28]: results = []

for rs in random_states:

    # Split dataset with different random_state
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=rs
    )

    # Feature scaling
    sc = StandardScaler()
    X_train = sc.fit_transform(X_train)
    X_test = sc.transform(X_test)

    # Train Logistic Regression
    classifier = LogisticRegression(random_state=0)
    classifier.fit(X_train, y_train)

    # Prediction
    y_pred = classifier.predict(X_test)

    # Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    TN, FP, FN, TP = cm.ravel()

    # Accuracy
    accuracy = (TP + TN) / (TP + TN + FP + FN)
```

```
# Store results
results.append([rs, TN, FP, FN, TP, accuracy])
```

```
In [29]: results_df = pd.DataFrame(
          results,
          columns=['Random State', 'TN', 'FP', 'FN', 'TP', 'Accuracy']
        )

          results_df
```

```
Out[29]:
```

	Random State	TN	FP	FN	TP	Accuracy
0	0	65	3	8	24	0.89
1	1	52	6	11	31	0.83
2	5	60	6	9	25	0.85
3	10	64	5	5	26	0.90
4	20	56	2	15	27	0.83

```
In [ ]:
```